README.md — Balloon Flight Display Live Sounding System

```
# Balloon Flight Display – Live Sounding System

This system enables on-demand atmospheric sounding retrieval for balloon flight
operations using live GPS position data.

It integrates:

- Flight Display (Expo / React Native)
- MQTT communication
- ICON-D2 model sounding fetch
- GeoJSON generation
- GitHub publishing
- Live visualization of wind drift, temperature, humidity

---

# System Architecture

FlightDisplay (ExpoGo)
        |
        | MQTT publish
        ▼
MQTT Broker (w471.gimmbh.com)
        |
        ▼
live_sounding_service.py
        |
        ├── fetch ICON-D2 sounding
        ├── convert to GeoJSON
        ├── copy latest file
        └── git push to GitHub
        |
        ▼
GitHub Repository (hlballon/hltemp)
        |
        ▼
FlightDisplay loads GeoJSON
        |
        ▼
Wind / Drift Visualization

---

# Current Status (Working)

The full system pipeline is operational:

✓ MQTT request triggered from button
✓ Position received correctly
✓ ICON-D2 sounding fetched
```

```
✓ Raw JSON created
✓ GeoJSON conversion successful
✓ latest_icon_sounding.geojson updated
✓ Git commit working
✓ Git push working
✓ Cyan temperature plotted
✓ Cyan humidity plotted
✓ Cyan wind speed plotted
✓ Cyan drift direction plotted

Wind direction now correctly converted from:

Wind-from → Drift-to

```js
direction = (rawDirection + 180) % 360;
```
```

Key Components

FlightDisplay (Expo App)

Main tasks:

- Publish MQTT sounding request
- Load GeoJSON sounding
- Plot:
    - Temperature
    - Humidity
    - Wind Speed
    - Drift Direction

Important fixes:

Supports both naming formats:

```
const rawDirection =
  props.windDirDeg ??
  props.wdir_deg ??
  props.direction ??
  null;

const direction =
  rawDirection != null
    ? (rawDirection + 180) % 360
    : null;

const speed =
  props.windSpeedKn ??
  props.wspd_kn ??
  props.speed ??
  null;
```

live_sounding_service.py

Main backend service.

Responsibilities:

- Subscribe to:

```
balloon/request_sounding
```

- Fetch ICON-D2 model
- Convert sounding → GeoJSON
- Copy:

```
latest_icon_sounding.geojson
```

- Commit + push to GitHub

Debounce protection:

```
MIN_INTERVAL_SEC = 300
```

Prevents repeated heavy downloads.

sounding_to_geojson.py

Converts model levels into GeoJSON features.

Required fields:

```
{
  "temp": 281.92,
  "dewpoint": 274.46,
  "gpheight": 10.0,
  "windSpeedKn": 1.6,
  "windDirDeg": 204.0
}
```

Data Flow

1. Button pressed
2. MQTT message sent
3. Python service receives position
4. ICON-D2 sounding downloaded
5. GeoJSON created
6. File pushed to GitHub
7. App reloads data

8. Drift visualization updated

Debugging Guide

MQTT Not Triggering

Check:

```
mosquitto_sub -h w471.gimmbh.com \
-t balloon/request_sounding \
-u lorMQTT \
-P l0rHBQND
```

Press button.

Expected:

```
{"Lat_deg":..., "Lon_deg":...}
```

Sounding Fetch Problems

Check:

```
Fetching ICON-D2...
Saved raw JSON...
```

If missing:

Check:

```
fetch_sounding.py
```

Missing Wind Data

Check:

```
python3
import json
data=json.load(open("latest_icon_sounding.geojson"))
print(data["features"][0]["properties"])
```

Must contain:

```
windSpeedKn
windDirDeg
```

Drift Direction Wrong

Verify conversion:

```
(rawDirection + 180) % 360
```

Without this:

Wind will be reversed.

Git Push Fails

Common error:

```
no upstream branch
```

Fix:

```
git branch -M main
git push --set-upstream origin main
```

Authentication fix:

Use:

- SSH key
  or
- GitHub Personal Access Token

Important Operational Notes

Debounce Protection

Repeated button presses within:

```
300 seconds
```

are ignored.

Log example:

```
⏳ Debounce active
```

This protects:

- server load
- network bandwidth

SSH Key Passphrase

Current behavior:

```
Enter passphrase for key...
```

For unattended operation:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Known Issues

Large Debug Output

Current debug logs may be verbose.

Recommended:

Reduce:

```
print(result)
```

To:

```
print("Fetch completed")
```

GitHub Caching Delay

Sometimes new GeoJSON takes:

```
10–60 seconds
```

to appear.

Workaround:

Toggle:

```
Hide → Fetch
```

in app.

Network Dependency

Requires:

- MQTT broker reachable
- GitHub reachable

- ICON data source reachable

Performance Notes

Typical fetch time:

```
15–20 seconds
```

Typical file size:

```
~50–150 KB GeoJSON
```

Typical workflow delay:

```
20–40 seconds total
```

Suggestions for Future Improvements

Multi-Model Support

Add:

- ICON-EU
- GFS fallback

Model Failover

If ICON fails:

Fallback to:

```
GFS
```

Local Cache

Avoid duplicate downloads when:

Same:

- position
- model run

Auto-Restart Service

Use:

```
systemd
```

to auto-start on boot.

Map Visualization

Add:

- drift path projection
- landing estimation

Recommended Production Setup

Run service via:

```
systemctl
```

Load SSH key automatically.

Disable verbose debug logging.

Monitor logs with:

```
journalctl -u sounding.service
```

Repository Structure

```
soundingFD/

├── data/
|   ├── raw/
|   └── geojson/
|
├── hltemp/
|   └── latest_icon_sounding.geojson
|
├── live_sounding_service.py
├── fetch_sounding.py
├── sounding_to_geojson.py
|
└── README.md
```

Final Status

System is:

```
OPERATIONAL
FIELD-READY
```

All core components verified working.

End-to-end pipeline functional.

```
---

# My Recommendation

Put this README in:

```text
/home/uhd/soundingFD/
```
```

and also into:

```
GitHub hlballon/hltemp
```